Lab 2 Extra Credit: A* vs Dynamically Weighted A*

Hannah Lee · CSCI 360

Setup

Weighted A* uses $f(n) = g(n) + w(n)\, h(n)$ with $w(n) = 1 + \varepsilon - \varepsilon\, d(n)/N$, $\varepsilon = 1$, $N = 2m$, and $d(n) = g(n)$ (number of flips so far). $h(n)$ is the same bad-pair heuristic as in Problem 2. Both searches use the same code otherwise: binary heap frontier, goal test when a node is popped, ties broken first-come-first-served, flips tried 1..m.

For m = 1..6 I ran both searches on **every** possible starting stack ($m! \cdot 2^m$: every order and every orientation). For m = 7 and 8 that is 645,120 and 10,321,920 stacks, and A* alone would take hours/days, so I used a fixed random sample of 2,000 stacks (seeded, same stacks for both searches). "Nodes visited" = number of nodes popped from the heap and expanded (every pop counts, the same way as the textbook's best-first search loop). A stack that is already sorted counts as 0 flips and 0 nodes.

Table 1: average number of flips

m	stacks	A*	Dynamically weighted A*	stacks where weighted used more flips
1	2	0.500	0.500	0
2	8	2.000	2.000	0
3	48	3.438	3.500	3
4	384	4.797	4.995	69
5	3840	6.117	6.466	1151
6	46080	7.399	7.915	19216
7	2000 (sample)	8.691	9.417	1101
8	2000 (sample)	9.851	10.836	1299

Table 2: average number of nodes visited

m	stacks	A*	Dynamically weighted A*
1	2	1.000	1.000
2	8	4.000	3.500
3	48	13.062	7.167
4	384	45.927	12.102
5	3840	169.287	18.082
6	46080	597.574	26.309
7	2000 (sample)	2074.492	33.934
8	2000 (sample)	6353.752	41.748

Table 3: running time, m = 1..5 (every stack)

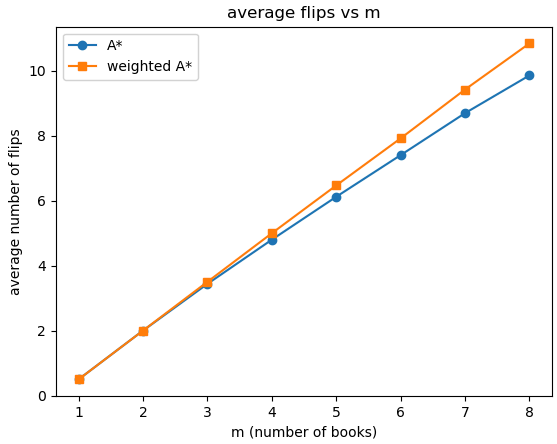
m	stacks	A* total (s)	weighted total (s)	A* avg per stack (ms)	weighted avg per stack (ms)
1	2	0.000022	0.000016	0.0112	0.0078
2	8	0.000095	0.000046	0.0118	0.0058
3	48	0.001020	0.000630	0.0212	0.0131
4	384	0.052283	0.015356	0.1362	0.0400
5	3840	2.414830	0.278670	0.6289	0.0726

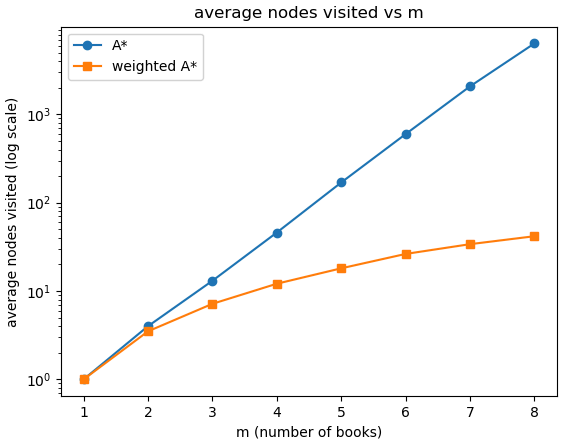
Table 4: average running time per stack, all m

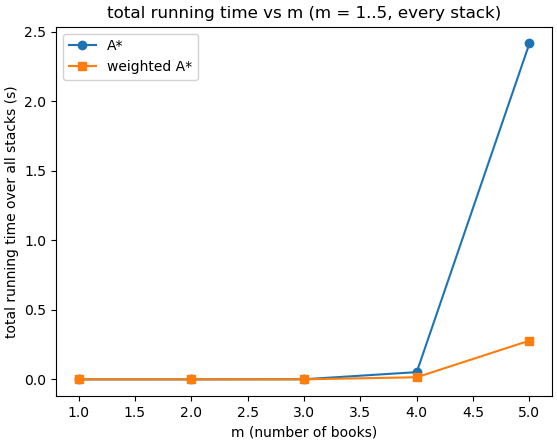
m	stacks	A* (ms)	weighted A* (ms)
1	2	0.0112	0.0078
2	8	0.0118	0.0058
3	48	0.0212	0.0131
4	384	0.1362	0.0400
5	3840	0.6289	0.0726
6	46080	3.2480	0.1487
7	2000 (sample)	16.3847	0.2477
8	2000 (sample)	68.6965	0.3945

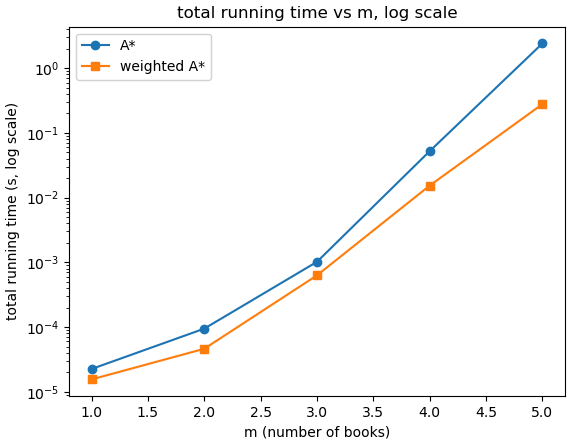
Timed with time.perf_counter around each search call. MacBook, Python, single run, so the very small numbers (m = 1, 2) are mostly noise.

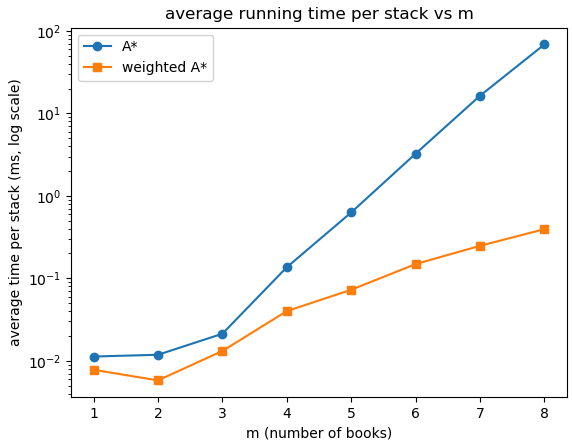
Figures


average flips vs m
A*
weighted A*
average number of flips
m (number of books)


average nodes visited vs m
A*
weighted A*
average nodes visited (log scale)
m (number of books)


total running time vs m (m = 1..5, every stack)
A*
weighted A*
total running time over all stacks (s)
m (number of books)


total running time vs m, log scale
A*
weighted A*
total running time (s, log scale)
m (number of books)


average running time per stack vs m
A*
weighted A*
average time per stack (ms, log scale)
m (number of books)

Analysis

Flips (solution quality). A* is optimal here because the heuristic is admissible: one flip only changes the one pair at the cut, so h can go down by at most 1 per flip and never overestimates. As a check, the A* averages for m = 1..5 (0.500, 2.000, 3.438, 4.797, 6.117) are exactly the BFS averages I got in Lab 1, and BFS is optimal for unit-cost flips. Dynamic weighting gives up a little of that: for $m \le 2$ the two are identical, and after that weighted A* needs slightly more flips on average (+0.06 at m = 3, +0.35 at m = 5, about +1.0 at m = 8, so roughly 2–10% more). The number of stacks where weighted A* is actually worse goes from 3/48 (m = 3) to 69/384 (m = 4) to 1151/3840 (m = 5) to 19216/46080 (m = 6), and more than half of the sampled stacks at m = 7 and 8 (1101/2000 and 1299/2000), so the bigger the stack, the more often the answer is not optimal. It is never wildly off though, because the weight never goes above $1 + \varepsilon = 2$, so the answer can't be worse than about 2× the optimal, and in practice it is far below that bound.

Nodes visited (search effort). This is where the two really split. A*'s average nodes grow by a factor of about 3–4 every time m goes up by one (169.3 → 597.6 → 2074.5 → 6353.8 for m = 5..8), which is exponential. Weighted A* grows by only about 6–8 nodes per extra book (18.1, 26.3, 33.9, 41.7), which looks closer to linear. At m = 8 weighted A* expands about 150× fewer nodes. The reason: this heuristic is pretty weak (it is at most m – 1 while the real answer is often more than m flips), so with w = 1 lots of states end up with the same f value and A* has to expand a big "flat" region before it is sure it has the best path. With w close to 2 near the start, the h part dominates f, so the search behaves almost greedily and dives toward stacks with fewer bad pairs. As d(n) grows, w(n) drops toward 1 and the search behaves more like plain A* for the last steps. That's exactly the "go somewhere first, then fine-tune" idea from the problem description. It is a similar idea to a decreasing schedule like the temperature in simulated annealing: behave one way early, then gradually switch to more careful behavior. If a path ever goes deeper than N = 2m, the code clamps w(n) at 1 so it never drops below plain A*.

Running time. Running time follows the node counts. For m = 1..5 over every stack, A*'s total time goes 0.00002 s → 0.0001 s → 0.001 s → 0.052 s → 2.41 s, multiplying by about 45–50× per extra book from m = 3 on, and the log-scale plot is close to a straight line (bending upward), so the growth is exponential in m. Two things stack up: the number of starting stacks grows like $m! \cdot 2^m$ (×2(m+1) going from m to m+1), and the time per stack also grows exponentially for A* (about ×4–6.5 per step: 0.02 → 0.14 → 0.63 → 3.25 → 16.4 → 68.7 ms). Weighted A*'s total time also grows exponentially (0.0006 s → 0.015 s → 0.28 s for m = 3..5) because the number of stacks does, but its time per stack only grows by about 1.6–2× per extra book from m = 4 on (0.07 ms at m = 5 vs 0.39 ms at m = 8, compared with 0.63 ms vs 69 ms for A*). This is why I could not run A* on every stack for m = 7 and 8 (hours to days), while weighted A* would have been doable (roughly an hour for all 10 million stacks at m = 8).

Takeaway. A* with this heuristic always finds the shortest flip sequence, but its effort grows exponentially with the stack size. Dynamically weighted A* trades a small loss in optimality (about one extra flip on average at m = 8, and a guaranteed bound of 2×) for a huge drop in nodes visited and running time. For small stacks plain A* is fine; for bigger stacks the weighted version is the practical choice.

Code: weighted_a_star_search (from lab2.py)

```
# how many states got popped and expanded in the last search
# (only for the extra credit tables)
nodes_visited = 0


def weighted_a_star_search(stack, epsilon=None, N=1):
    # Weighted A* is extra credit

    flip_sequence = []
    # --- v ADD YOUR CODE HERE v --- #
    # same as a_star_search, only the priority changes:
    #   f = g + push_factor * h
    #   push_factor = 1 + epsilon - epsilon * depth / N
    # big push_factor early (greedy), shrinks to 1 deeper in (plain A*)
    global nodes_visited
    nodes_visited = 0

    num_books = stack.num_books

    # lab says epsilon = 1 and N = 2m
    # default N=1 would make push_factor negative, so swap it for 2m
    if epsilon is None:
        epsilon = 1
    if N is None or N == 1:
        N = 2 * num_books

    start_order = []
    start_face = []
    for i in range(0, num_books):
        start_order.append(int(stack.order[i]))
        start_face.append(int(stack.orientations[i]))
    start_state = (tuple(start_order), tuple(start_face))

    # reached_goal: book i at spot i, all facing up
    reached_goal = True
    for i in range(0, num_books):
        if start_order[i] != i:
            reached_goal = False
        if start_face[i] != 1:
            reached_goal = False

    if reached_goal == True:
        return flip_sequence

    # count_bad_pairs: same heuristic as A*
    mess_count = 0
    for i in range(0, num_books - 1):
        top_book = start_order[i]
        below_book = start_order[i + 1]
        top_face = start_face[i]
        below_face = start_face[i + 1]

        pair_is_good = False

        if top_face == 1 and below_face == 1:
            if below_book - top_book == 1:
                pair_is_good = True
        elif top_face == 0 and below_face == 0:
            if top_book - below_book == 1:
                pair_is_good = True

        if pair_is_good == False:
            mess_count = mess_count + 1
    start_h = mess_count
```

```
    # push_factor at depth 0 -> 1 + epsilon
    push_factor = 1 + epsilon - (epsilon * 0 / N)
    if push_factor < 1:
        push_factor = 1
    start_f = 0 + push_factor * start_h

    waiting_line = []
    arrival_order = 0

    best_count_at = {}
    best_count_at[start_state] = 0

    came_from = {}
    came_from[start_state] = None

    heappush(waiting_line, (start_f, arrival_order, 0, start_state))

    while len(waiting_line) > 0:
        popped = heappop(waiting_line)
        steps_taken = popped[2]
        stack_rightnow = popped[3]

        nodes_visited = nodes_visited + 1

        order_now = stack_rightnow[0]
        face_now = stack_rightnow[1]

        # goal test on pop
        reached_goal = True
        for i in range(0, num_books):
            if order_now[i] != i:
                reached_goal = False
            if face_now[i] != 1:
                reached_goal = False

        if reached_goal == True:
            rewind_list = []
            walker = stack_rightnow
            while came_from[walker] is not None:
                previous_state = came_from[walker][0]
                used_cut = came_from[walker][1]
                rewind_list.append(used_cut)
                walker = previous_state

            i = len(rewind_list) - 1
            while i >= 0:
                flip_sequence.append(rewind_list[i])
                i = i - 1

            return flip_sequence

        for cut_spot in range(1, num_books + 1):
            # do_one_flip
            new_order = []
            new_face = []

            i = cut_spot - 1
            while i >= 0:
                new_order.append(order_now[i])
                new_face.append(1 - face_now[i])
                i = i - 1

            i = cut_spot
            while i < num_books:
                new_order.append(order_now[i])
                new_face.append(face_now[i])
                i = i + 1
```

```
            after_flip = (tuple(new_order), tuple(new_face))
            new_steps = steps_taken + 1

            # weighted h isn't consistent, so a state may come back cheaper
            # -> strictly cheaper path re-opens it
            should_add = False
            if after_flip not in best_count_at:
                should_add = True
            elif new_steps < best_count_at[after_flip]:
                should_add = True

            if should_add == True:
                best_count_at[after_flip] = new_steps
                came_from[after_flip] = (stack_rightnow, cut_spot)

                # count_bad_pairs again
                mess_count = 0
                for i in range(0, num_books - 1):
                    top_book = new_order[i]
                    below_book = new_order[i + 1]
                    top_face = new_face[i]
                    below_face = new_face[i + 1]

                    pair_is_good = False

                    if top_face == 1 and below_face == 1:
                        if below_book - top_book == 1:
                            pair_is_good = True
                    elif top_face == 0 and below_face == 0:
                        if top_book - below_book == 1:
                            pair_is_good = True

                    if pair_is_good == False:
                        mess_count = mess_count + 1
                new_h = mess_count

                # dynamic weight at this depth
                push_factor = 1 + epsilon - (epsilon * new_steps / N)
                # past depth N it would drop below 1 -> just stay plain A*
                if push_factor < 1:
                    push_factor = 1

                # f = g + push_factor * h
                new_f = new_steps + push_factor * new_h

                arrival_order = arrival_order + 1
                heappush(waiting_line, (new_f, arrival_order, new_steps, after_flip))

    return flip_sequence

    # ----------------------------- #
```

Code: lab2_extra_credit.py (experiment + tables + figures)

```
# Lab 2 Extra Credit (Problem 3): A* vs dynamically weighted A*
# this is for extra credit not the main hw
# for each m, run both searches on the stacks and record
#   - average flips
#   - average nodes visited
#   - running time
# m = 1..6 -> every possible stack (m! * 2^m)
# m = 7, 8 -> too many stacks (A* alone would take hours / days)
#            so a fixed random sample, same sample for both searches

# import
import itertools
import random
import time
import json
import matplotlib.pyplot as plt
import lab2
from lab2 import TextbookStack


# how many random stacks for the big m
SAMPLE_SIZE = 2000
# biggest m where we still try every stack
LAST_FULL_M = 6


if __name__ == "__main__":
    m_values = [1, 2, 3, 4, 5, 6, 7, 8]

    all_results = []

    for m in m_values:
        # make the list of starting stacks
        stacks_to_try = []
        if m <= LAST_FULL_M:
            # every (order, orientations) pair, m! * 2^m total
            for order in itertools.permutations(range(0, m)):
                for orientations in itertools.product([0, 1], repeat=m):
                    stacks_to_try.append((list(order), list(orientations)))
            how_chosen = "all"
        else:
            # random sample, seeded so it's the same every run
            random.seed(360 + m)
            for k in range(0, SAMPLE_SIZE):
                order = []
                for i in range(0, m):
                    order.append(i)
                random.shuffle(order)

                orientations = []
                for i in range(0, m):
                    orientations.append(random.randint(0, 1))

                stacks_to_try.append((order, orientations))
            how_chosen = "sample"

        num_stacks = len(stacks_to_try)
        print("m =", m, ":", num_stacks, "stacks (" + how_chosen + ")")

        # ---- run A* on every stack ----
        astar_total_flips = 0
        astar_total_nodes = 0
        astar_total_time = 0.0
        astar_lengths = []
```

```
    for order, orientations in stacks_to_try:
        one_stack = TextbookStack(order, orientations)

        start_clock = time.perf_counter()
        flip_sequence = lab2.a_star_search(one_stack)
        end_clock = time.perf_counter()

        # sanity check: it really got sorted
        if lab2.apply_sequence(one_stack, flip_sequence).check_ordered() == False:
            print("  !! A* not sorted:", order, orientations)

        astar_total_flips = astar_total_flips + len(flip_sequence)
        astar_total_nodes = astar_total_nodes + lab2.nodes_visited
        astar_total_time = astar_total_time + (end_clock - start_clock)
        astar_lengths.append(len(flip_sequence))

    print("  A* done")

    # ---- run weighted A* on every stack (same thing) ----
    weighted_total_flips = 0
    weighted_total_nodes = 0
    weighted_total_time = 0.0
    weighted_lengths = []

    for order, orientations in stacks_to_try:
        one_stack = TextbookStack(order, orientations)

        start_clock = time.perf_counter()
        flip_sequence = lab2.weighted_a_star_search(one_stack)
        end_clock = time.perf_counter()

        if lab2.apply_sequence(one_stack, flip_sequence).check_ordered() == False:
            print("  !! weighted A* not sorted:", order, orientations)

        weighted_total_flips = weighted_total_flips + len(flip_sequence)
        weighted_total_nodes = weighted_total_nodes + lab2.nodes_visited
        weighted_total_time = weighted_total_time + (end_clock - start_clock)
        weighted_lengths.append(len(flip_sequence))

    print("  weighted A* done")

    # how many stacks weighted A* solved with MORE flips than A*
    worse_count = 0
    for k in range(0, num_stacks):
        if weighted_lengths[k] > astar_lengths[k]:
            worse_count = worse_count + 1

    astar_result = {}
    astar_result["average_flips"] = astar_total_flips / num_stacks
    astar_result["average_nodes"] = astar_total_nodes / num_stacks
    astar_result["total_time"] = astar_total_time
    astar_result["average_time"] = astar_total_time / num_stacks

    weighted_result = {}
    weighted_result["average_flips"] = weighted_total_flips / num_stacks
    weighted_result["average_nodes"] = weighted_total_nodes / num_stacks
    weighted_result["total_time"] = weighted_total_time
    weighted_result["average_time"] = weighted_total_time / num_stacks

    one_row = {}
    one_row["m"] = m
    one_row["num_stacks"] = num_stacks
    one_row["how_chosen"] = how_chosen
    one_row["astar"] = astar_result
    one_row["weighted"] = weighted_result
    one_row["weighted_worse_count"] = worse_count
    all_results.append(one_row)
```

```
    # save numbers so the pdf can use them
    out_file = open("lab2_extra_credit_results.json", "w")
    json.dump(all_results, out_file, indent=2)
    out_file.close()

    # table 1: average flips
    print("")
    print("m | stacks | A* avg flips | weighted A* avg flips | weighted worse")
    for row in all_results:
        print(row["m"], "|", row["num_stacks"], "|",
              round(row["astar"]["average_flips"], 3), "|",
              round(row["weighted"]["average_flips"], 3), "|",
              row["weighted_worse_count"])

    # table 2: average nodes visited
    print("")
    print("m | stacks | A* avg nodes | weighted A* avg nodes")
    for row in all_results:
        print(row["m"], "|", row["num_stacks"], "|",
              round(row["astar"]["average_nodes"], 3), "|",
              round(row["weighted"]["average_nodes"], 3))

    # table 3: running time
    print("")
    print("m | A* total s | A* avg ms | weighted total s | weighted avg ms")
    for row in all_results:
        print(row["m"], "|",
              round(row["astar"]["total_time"], 6), "|",
              round(row["astar"]["average_time"] * 1000, 4), "|",
              round(row["weighted"]["total_time"], 6), "|",
              round(row["weighted"]["average_time"] * 1000, 4))

    # pull columns out for the plots
    astar_flips = []
    weighted_flips = []
    astar_nodes = []
    weighted_nodes = []
    astar_avg_ms = []
    weighted_avg_ms = []
    for row in all_results:
        astar_flips.append(row["astar"]["average_flips"])
        weighted_flips.append(row["weighted"]["average_flips"])
        astar_nodes.append(row["astar"]["average_nodes"])
        weighted_nodes.append(row["weighted"]["average_nodes"])
        astar_avg_ms.append(row["astar"]["average_time"] * 1000)
        weighted_avg_ms.append(row["weighted"]["average_time"] * 1000)

    # graded part is m = 1..5, total time over every stack
    graded_m = []
    astar_total_s = []
    weighted_total_s = []
    for row in all_results:
        if row["m"] <= 5:
            graded_m.append(row["m"])
            astar_total_s.append(row["astar"]["total_time"])
            weighted_total_s.append(row["weighted"]["total_time"])

    # figure 1: average flips
    plt.figure()
    plt.plot(m_values, astar_flips, marker="o", label="A*")
    plt.plot(m_values, weighted_flips, marker="s", label="weighted A*")
    plt.xlabel("m (number of books)")
    plt.ylabel("average number of flips")
    plt.title("average flips vs m")
    plt.legend()
    plt.savefig("lab2_average_flips.png")

    # figure 2: average nodes visited (log scale, it grows fast)
    plt.figure()
```

```
    plt.plot(m_values, astar_nodes, marker="o", label="A*")
    plt.plot(m_values, weighted_nodes, marker="s", label="weighted A*")
    plt.yscale("log")
    plt.xlabel("m (number of books)")
    plt.ylabel("average nodes visited (log scale)")
    plt.title("average nodes visited vs m")
    plt.legend()
    plt.savefig("lab2_average_nodes.png")

    # figure 3: total running time for m = 1..5 (the graded part)
    plt.figure()
    plt.plot(graded_m, astar_total_s, marker="o", label="A*")
    plt.plot(graded_m, weighted_total_s, marker="s", label="weighted A*")
    plt.xlabel("m (number of books)")
    plt.ylabel("total running time over all stacks (s)")
    plt.title("total running time vs m (m = 1..5, every stack)")
    plt.legend()
    plt.savefig("lab2_total_time.png")

    # figure 4: same thing on log scale -> straight line = exponential
    plt.figure()
    plt.plot(graded_m, astar_total_s, marker="o", label="A*")
    plt.plot(graded_m, weighted_total_s, marker="s", label="weighted A*")
    plt.yscale("log")
    plt.xlabel("m (number of books)")
    plt.ylabel("total running time (s, log scale)")
    plt.title("total running time vs m, log scale")
    plt.legend()
    plt.savefig("lab2_total_time_log.png")

    # figure 5: average time per stack, all m
    plt.figure()
    plt.plot(m_values, astar_avg_ms, marker="o", label="A*")
    plt.plot(m_values, weighted_avg_ms, marker="s", label="weighted A*")
    plt.yscale("log")
    plt.xlabel("m (number of books)")
    plt.ylabel("average time per stack (ms, log scale)")
    plt.title("average running time per stack vs m")
    plt.legend()
    plt.savefig("lab2_average_time.png")
```